Exp No: 01
Date: 15/7/26

Basic Networking Commands in Linux and Windows operating System

Aim: To study and understand basic Network commands in Linux and Windows Operating System.

```
student@linux: ~/ThirdYearIT
File  Edit  View  Search  Terminal  Help
student@linux:~$ mkdir ThirdYearIT
student@linux:~$ pwd
/home/student
student@linux:~$ cd ThirdYearIT
student@linux:~/ThirdYearIT$ pwd
/home/student/ThirdYearIT
student@linux:~/ThirdYearIT$ touch rec.txt
student@linux:~/ThirdYearIT$ echo "Welcome to Third Year IT\nLinux commands are very useful." > rec
student@linux:~/ThirdYearIT$ echo "Welcome to Third Year IT\nLinux commands are very useful."
Welcome to Third Year IT
Linux commands are very useful.
student@linux:~/ThirdYearIT$ cat rec.txt
Welcome to Third Year IT
Linux commands are very useful.
student@linux:~/ThirdYearIT$ grep Linux rec.txt
Linux commands are very useful.
student@linux:~/ThirdYearIT$ wc -l rec.txt
2 rec.txt
student@linux:~/ThirdYearIT$ wc -w rec.txt
8 rec.txt
student@linux:~/ThirdYearIT$ wc -c rec.txt
52 rec.txt
student@linux:~/ThirdYearIT$ wc -m rec.txt
52 rec.txt
student@linux:~/ThirdYearIT$ wc -L rec.txt
31 rec.txt
student@linux:~/ThirdYearIT$
```

```
student@linux: ~/ThirdYearIT
File  Edit  View  Search  Terminal  Help
student@linux:~/ThirdYearIT$ hostname
linux
student@linux:~/ThirdYearIT$ group
student adm cdrom sudo dip plugdev lpadmin lxd sambashare
student@linux:~/ThirdYearIT$ nslookup google.com
Server:         8.8.8.8
Address:        8.8.8.8#53

Non-authoritaive answer:
Name:    google.com
Address: 142.250.195.78
student@linux:~/ThirdYearIT$ arp -n
Address                  HWtype  HWaddress           Flags Mask      Iface
192.168.1.1              ether   4c:6f:8c:1a:2b:3c   C               eth0
192.168.1.105            ether   00:0c:29:7d:6e:1a   C               eth0
student@linux:~/ThirdYearIT$ traceroute google.com
traceroute to google.com (142.250.195.78), 30 hops max, 60 byte packets
 1  192.168.1.1      1.192 ms   1.135 ms   1.128 ms
 2  10.0.0.1         7.895 ms   7.812 ms   7.749 ms
 3  172.16.0.1      14.532 ms  14.421 ms  14.305 ms
 4  142.250.195.78  20.894 ms  20.771 ms  20.658 ms

student@linux:~/ThirdYearIT$
```

Result:

Thus, Basic Networking Commands in Linux and Windows Operating System is executed successfully

Exp - 02
Date - 16/7/26 STUDY OF DIFFERENT TYPES OF NETWORK CABLES

~~Exp No. 02 Learning and Assignment of IP~~
~~Date: 16/7/26 Address manually to computers~~

AIM:
To study the different types of network cables

a) Understanding different types of network cables

The different types of cables used in networking

1. Unshielded Twisted Pair (UTP) cable
2. shielded Twisted Pair (STP) cable
3. coaxial cable:
4. Fibre optic cable

Cable Type	Category	Maximum data Transmission	Advantages/ Disadvantages	Applications
UTP	category 3 category 5 category 5e	10bps upto 100mbps 1gbps	Advantages -> Cheaper -> Easy to install Disadvantages -> more prone do EMI & noise	10 Base-T Ethernet Fast Ethernet
STP	Category 6, 6a	10Gbps	Advantages -> shielded -> Faster than UTP -> Less to noise	Gigabit - Ethernet
Coaxial Cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantage: -> High bandwidth -> Versatile Disadvantage: -> Limited Distance -> cost -> Size is bulky	Speed of signal is 500 m, Television Network

Cable type	category	max data transmission	Advantage/ Disadvantage	Application/ use
Fibre optic cable	single mode, multi mode	100 gbps	Advantage: → High speed → High bandwidth → High security Disadvantage → Expensive → Requires skilled installers	Maximum distance of fibre optics cable is 100 meters

Result:

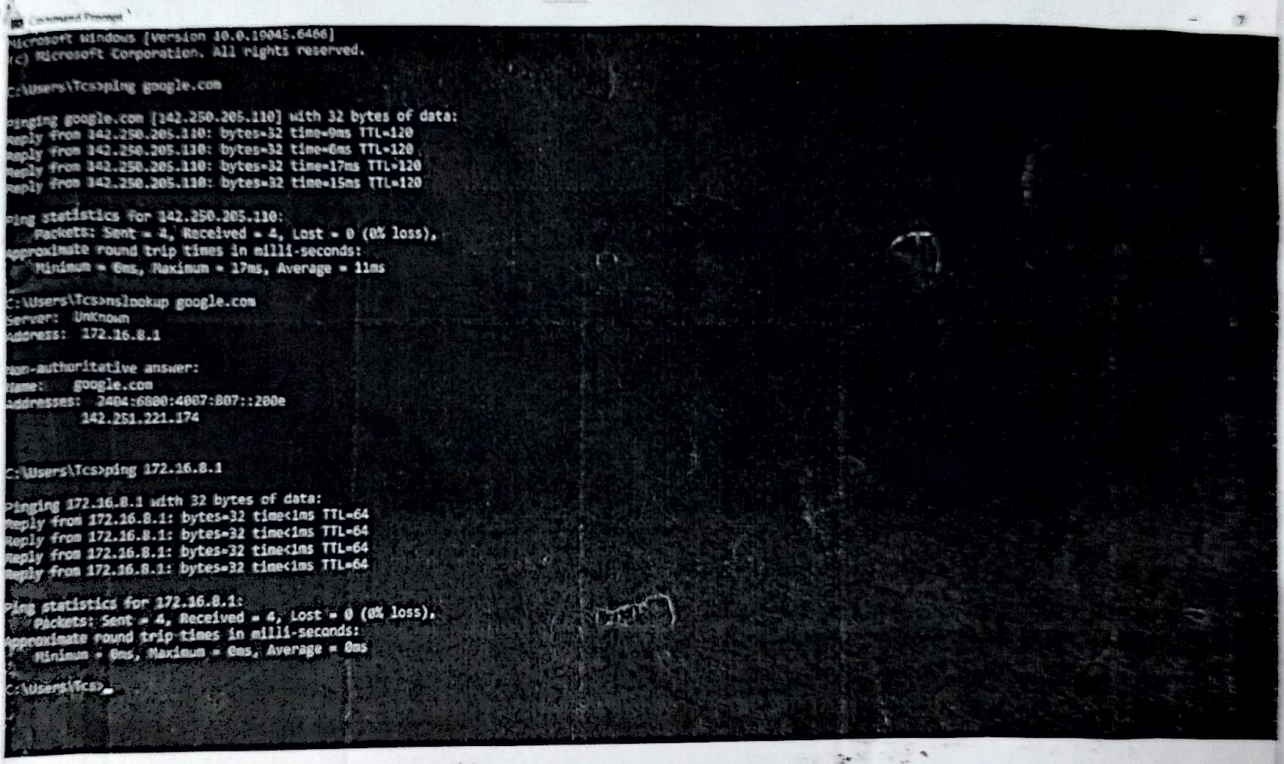
Thus, the different types of networking cables were studied and verified

Exp No: 03
Date: 18/7/26 Develop a customised ping command to test the Server Connectivity

AIM:

To develop and use a customized ping command to test the connectivity between a client and server.

3/26, 6:51 PM 301.jpg



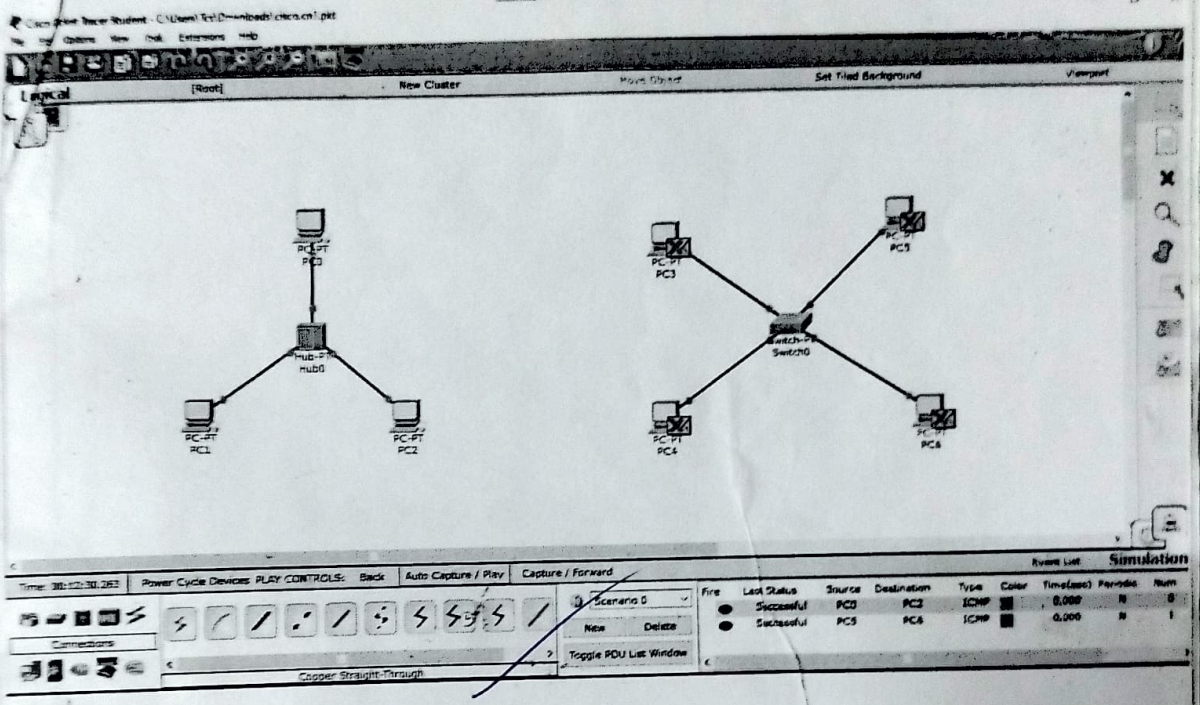
Result:

Thus, the ping Command to test the Server Connectivity is developed successfully.

Exp No: 4 Packet Tracer tool Installation and
Date: 22/7/26 User Interface Overview

AIM:
To install cisco packet tracer and study its user interface and basic features.

3:26, 6:5 PM 300.jpg



Result:
Thus, the packet tracer tool Installation of user interface overview is executed succesfully

Exp No : 05
Date : 23/7/24

Implementing Echo using Python

Aim:

To implement echo using python programming

```
import socket

HOST = '127.0.0.1'  # localhost
PORT = 1234

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print("Server is waiting for connection...")

conn, addr = server.accept()
print("Connected by", addr)

while True:
    data = conn.recv(1024).decode()

    if not data or data.lower() == "quit":
        print("Connection closed.")
        break

    print("Client:", data)
    conn.send(data.encode())  # Echo back the same message

conn.close()
server.close()
```



```
import socket

HOST = '127.0.0.1'
PORT = 1234

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

while True:
    message = input("Enter message: ")

    client.send(message.encode())

    if message.lower() == "quit":
        break

    data = client.recv(1024).decode()
    print("Server:", data)

client.close()
```



```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "help" above for more information.

>>> ========== RESTART: C:\Users\HEC\Downloads\client.py ==========
Enter message: HAI
Server: HAI
Enter message: HAI
Server: HAI
Enter message:
```



```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "help" above for more information.

>>> ========== RESTART: C:\Users\HEC\Downloads\server.py ==========
Server is waiting for connection...
Connected by: ('127.0.0.1', 57474)
Client: HAI
Client: HAI
```



Result

Thus, Implementing echo using Python is executed successfully.

Exp No : 06

Date : 30/7/26

Byte Stuffing using Python

Aim : To implement Byte Stuffing using Python

byte_stuffing.py - C:/Users/REC/AppData/Local/Programs/Python/Python310/byte_stuffing.p

File Edit Format Run Options Window Help

```
# Byte Stuffing with Binary Representation

flag = input("Enter Flag Character: ")
escape = input("Enter Escape Character: ")
data = input("Enter Data: ")

stuffed = ""

for ch in data:
    if ch == flag or ch == escape:
        stuffed += escape
    stuffed += ch

frame = flag + stuffed + flag

print("\nOriginal Data       :", data)
print("Stuffed Data        :", stuffed)
print("Transmitted Frame   :", frame)

# Function to print binary values
def show_binary(text):
    for ch in text:
        print(ch, "->", format(ord(ch), '08b'))

print("\nBinary of Flag:")
show_binary(flag)

print("\nBinary of Escape:")
show_binary(escape)

print("\nBinary of Original Data:")
show_binary(data)

print("\nBinary of Stuffed Data:")
show_binary(stuffed)

print("\nBinary of Transmitted Frame:")
show_binary(frame)
```

IDLE Shell 3.10.6

File Edit Shell Debug Options Window Help

```
>>>
= RESTART: C:/Users/REC/AppData/Local/Programs/Python/Python310/byte_stuffing.py
Enter Flag Character: F
Enter Escape Character: E
Enter Data: AEFB

Original Data       : AEFB
Stuffed Data        : AEEEFB
Transmitted Frame   : FAEEEFBF

Binary of Flag:
F -> 01000110

Binary of Escape:
E -> 01000101

Binary of Original Data:
A -> 01000001
E -> 01000101
F -> 01000110
B -> 01000010

Binary of Stuffed Data:
A -> 01000001
E -> 01000101
E -> 01000101
E -> 01000101
F -> 01000110
B -> 01000010

Binary of Transmitted Frame:
F -> 01000110
A -> 01000001
E -> 01000101
E -> 01000101
E -> 01000101
F -> 01000110
B -> 01000010
F -> 01000110
>>>
```

Ln: 42 Col: 0

Result:

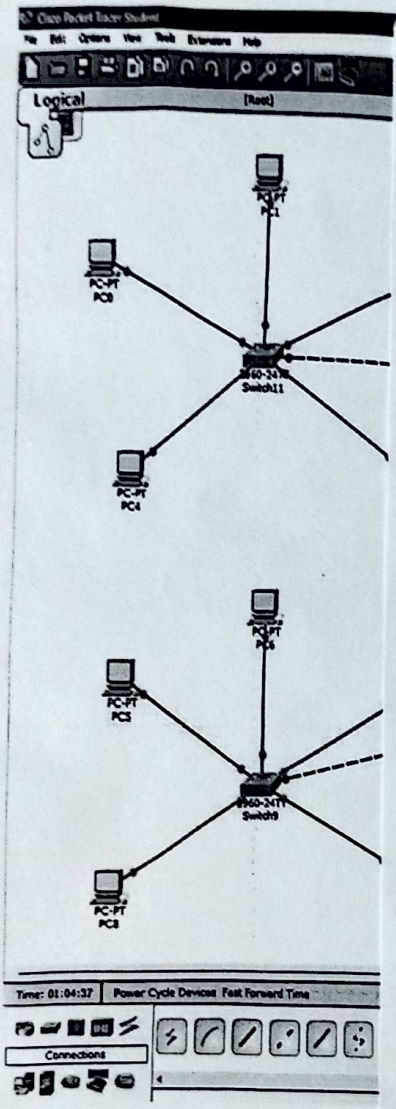
Thus Byte stuffing using Python is executed Successfully.

Date-19/8/24

Implementation of Subnetting in cisco packet tracer simulator

Aim:

Implementation of Subnetting in cisco packet tracer simulator



Result:

Thus, its implements Subnetting in cisco packet tracer simulator

Exp: 8
Date: 9/9/26

Client - Server python Socket programming

Aim:

To implement client - server communication using python Socket programming.

client.py - C:/Users/RSC/client.py (3.12.10)

```
File Edit Format Run Options Window Help
import socket
import threading

SERVER_IP = "127.0.0.1"
PORT = 5001

def receive_messages(sock):
    while True:
        try:
            message = sock.recv(1024).decode()

            if not message:
                break

            print(f"\nServer: {message}")
            print("You: ", end="", flush=True)

        except ConnectionResetError:
            break

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_IP, PORT))

print("Connected to server.")

threading.Thread(
    target=receive_messages,
    args=(client,),
    daemon=True
).start()

while True:
    message = input("You: ")

    if message.lower() == "exit":
        break

    client.sendall(message.encode())

client.close()
```

server.py - C:/Users/RSC/server.py (3.12.10)

```
File Edit Format Run Options Window Help
import socket
import threading

HOST = "0.0.0.0"
PORT = 5001

def receive_messages(conn):
    while True:
        try:
            message = conn.recv(1024).decode()

            if not message:
                break

            print(f"\nClient: {message}")
            print("You: ", end="", flush=True)

        except ConnectionResetError:
            break

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print(f"Server listening on port {PORT}...")

conn, addr = server.accept()
print(f"Client connected: {addr}")

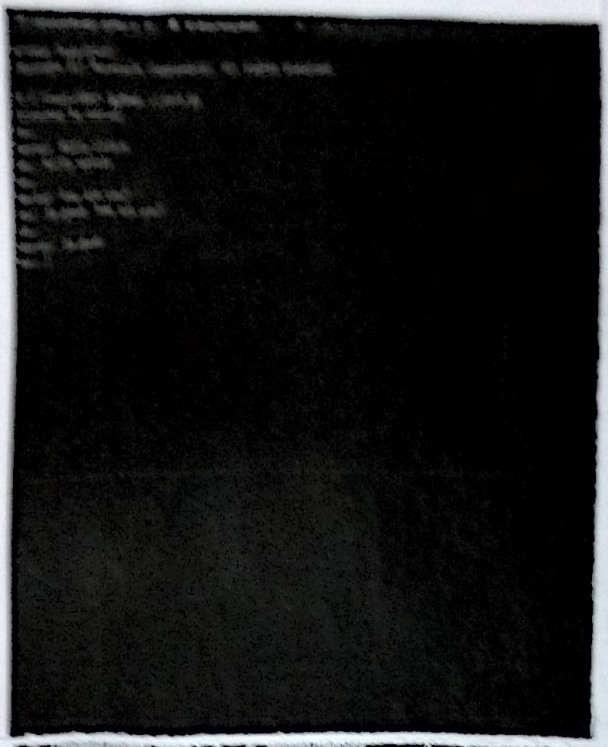
threading.Thread(
    target=receive_messages,
    args=(conn,),
    daemon=True
).start()

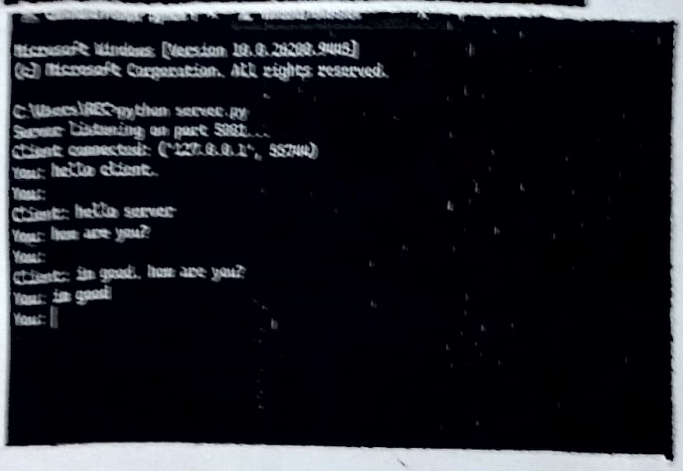
while True:
    message = input("You: ")

    if message.lower() == "exit":
        break

    conn.sendall(message.encode())

conn.close()
server.close()
```





Result :

Thus, to implement client-sower communication using python socket programming

Exp: 9
Date: 14/9/24

Distance vector Routing

Aim:

To implement Distance vector Routing

Command Prompt - python

```
...     net.display_tables()

Simulating Distance Vector Routing Protocol...
Network converged in 3 iterations.

--- Routing Table for Router A ---
Destination     Cost        Next Hop
A               0           A
B               2           B
C               3           B
D               5           B

--- Routing Table for Router B ---
Destination     Cost        Next Hop
A               2           A
B               0           B
C               1           C
D               3           C

--- Routing Table for Router C ---
Destination     Cost        Next Hop
A               7           A
B               1           B
C               0           C
D               2           D

--- Routing Table for Router D ---
Destination     Cost        Next Hop
A               inf         -
B               4           B
```

Result:

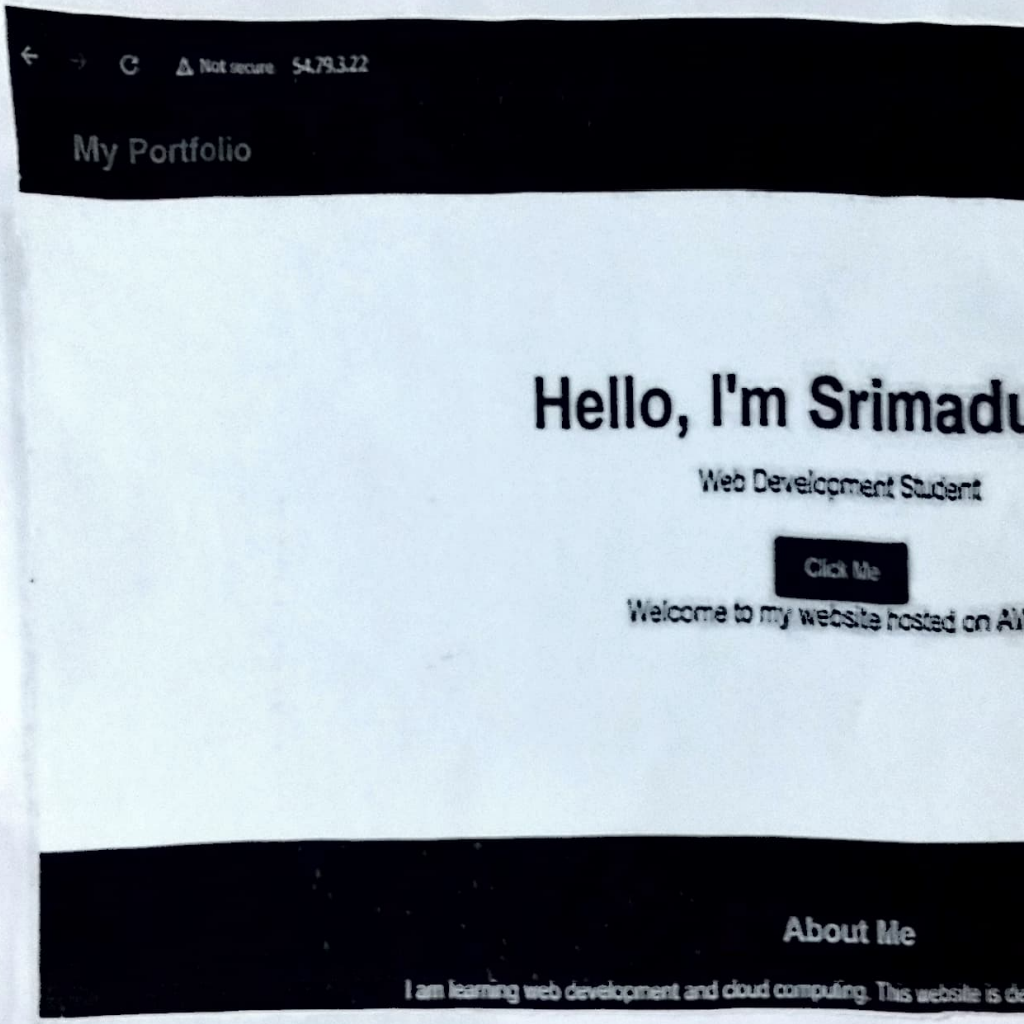
Thus, to implement Distance Vector Routing

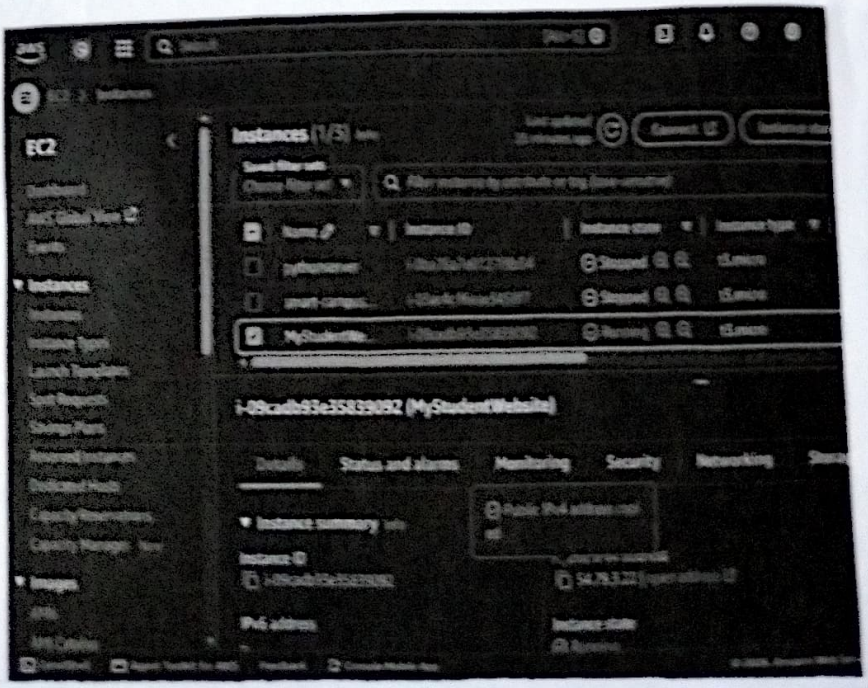
Exp: 10

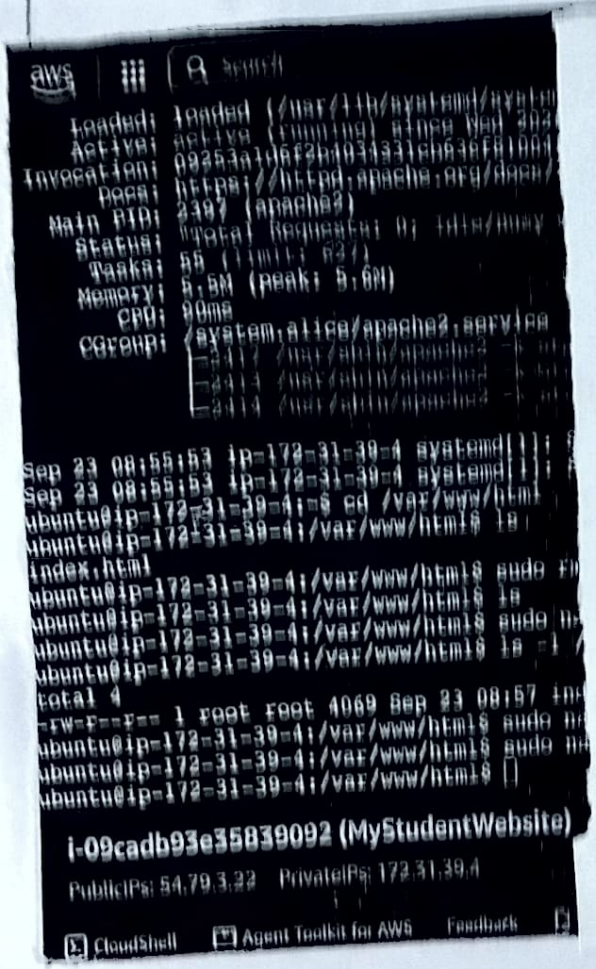
Date : 23/9/26 Deploying a web application on AWS Server

Aim :

To deploy a web application on AWS server







Result:

Thus, to deploy a web application on Aws Server.

Exp - 11
Date - 24/8/26

Experiment on Packet Capture Tool - Wireshark

AIM:

Capturing an analysis packets using wireshark

1) AIM:

Create a filter to display only TCP/UDP packets, inspects the packets and provide the flow graph

PROCEDURE:

* Select LAN connection in wireshark.
* Go to Capture → option
* Select Stop capture automatically after 100 packets
* Save the packets

2) AIM:

Create a filter to display only ARP packets and inspect the packets and

PROCEDURE:

* Go to capture
* set Stop capture automatically after 100 packets
* Then click Start capture
* Save the packets

3) AIM:

Create a filter to display only DNS packets and provide the flow graph

PROCEDURE

* Go to capture - option
* select Stop capture automatically after 100 packets
* Then click start capture
* Search DNS packets
* save the packets

4) AIM:

Create a filter to display only HTTP packet and repeat the packets

PROCEDURE:

* Select LAN connection in wireshark
* Go to capture → Option
* Then click start capture
* Save packets

5, Aim

To create a filter to display only IP/ICMP packets and inspect the packets

PROCEDURE

* Select a LAN connection in wireshark
* Go to capture -> option
* Click start capture
* Search for ICMP/IP packets
* Save the packets

6) Aim:

Create a filter to display only SMTP packets and inspect the packets.

PROCEDURE:

* Select LAN connection in wireshark
* Go to capture -> option
* Select the stop capture automatically
* Save the packets.

Result:

This network packets were successfully captured and analysed using wireshark with different protocol filters.

Exp. - 12
Date - 20/9/24

Discovery of live host using ARP, ICMP, TCP, UDP

AIM:

To discover live host in a network using Nmap scans such as ARP, ICMP, TCP & UDP on kaylinux platform

OBJECTIVE:

To understand how nmap identifies active systems before performing further network scanning.

SOFTWARE:

Nmap
Kaylinux Nmap Zenmap
Computer with network access

THEORY:

Nmap is a free and open-source network scanning tool used to discover live hosts. Nmap uses different live hosts using different protocols

1) ARP scan - uses ARP scans request to find hosts on same subnet
2) ICMP scan - use icmp request and replies for identifies active host
3) TCP scan - use TCP requests and packets to specified ports and check the response
4) UDP scan - Send UDP packets & can identify an active host through an ICMP port

OBSERVATION:

-> ARP request can discover hosts within same subnet
-> ARP cannot cross routers because it operates at link layer
-> An ARP ping generally requires a privileged account
-> UDP ping can identify an active host when an icmp port unreachable is received
-> The -sn option allows Nmap to perform host discovery without conducting a port scan

Result:

Thus, live host discovery using Nmap was successfully studied and performed. The experiment helped [illegible] active hosts can be identified [illegible] further network [illegible]

Sliding Window Protocol

AIM:

To write a program to implement flow control at data link layer using the sliding window protocol and simulate the flow of frames from one node to another.

SENDER CODE:

```
message = input ("Enter the message: ")
window_size = int (input ("Enter window size: "))
frames = []
for i, ch in enumerate (message):
    frames.append ((i+1, ch))
print ("\n Frames created: ")
for no, data in frames:
    print (f"Frame {no}: {data}")
with open ("sender_Buffer.txt", "w") as file:
    for no, data in frames:
        file.write (f"{no} {data}\n")
print ("\n Frames stored in Sender_Buffer.txt")
base = 0
while base < len (frames):
    print ("\n Current window: ")
    for i in range (base, min (base + window_size,
        len (frames))):
        print (f"[{frames[i][0]}: {frames[i][1]}]",
            end = " ")
    try:
        with open ("Receiver_Buffer.txt", "r") as file:
            response = file.readline().strip()
    except FileNotFoundError:
        response = ""
    if response.startswith ("ACK"):
        ack = int (response.split()[1])
        print (f"ACK {ack} received")
        base = ack
        open ("Receiver_Buffer.txt", "w").close()
```

```
Receiver.py
Expected_frame = 1
while True:
    try:
        with open("Receiver_Buffer.txt", "r") as file:
            frames = file.readlines()
    except FileNotFoundError:
        print("Receiver_Buffer.txt not found. Waiting...")
        time.sleep(1)
        continue
    if not frames:
        print("No frames available")
        break

    print("\nReceiver received frame status:")
    for line in frames:
        parts = line.split()
        if len(parts) != 3:
            continue
        frame_no = int(parts[0])
        state = parts[1]
        print(f"Frame {frame_no} {state}")
        if frame_no == expected_frame:
            print(f"Frame {frame_no} received")
            with open("Receiver_Buffer.txt", "w") as ack_file:
                ack_file.write(f"ACK {frame_no}")
            print(f"ACK {frame_no} sent")
            Expected_frame += 1
    else:
        print("Wrong frame received")
        with open("Receiver_Buffer.txt", "w")
            as ack_file:
            ack_file.write(f"NACK {expected_frame}
        break                    frame sent")
if expected_frame > len(frames):
    print("All frames received")
    break
input("Press Enter to check next frame")
```

Input :

Enter the message : HELLO

Enter window size : 3

OUTPUT :

Frames created :

Frame 1 : H

Frame 2 : E

Frame 3 : L

Frame 4 : L

Frame 5 : O

Frames stored in sender_buffer.txt

Current window:

[1 : H] [2 : E] [3 : L]

Frames sent :

Run Receiver program and press ENTER to continue

RECEIVER OUTPUT :

Frames received from sender :

Frame 1 : H

Frame 1 received correctly

ACK 1 sent

Result :

The Sliding Window protocol was successfully implemented using Python and the data transmission and acknowledgement process was verified successfully.

Exp: 11
Date - 21/10/24

Implementing Packet Sniffing Using Raw Sockets

AIM:

To implement packet sniffing using RAW sockets in python

ALGORITHM:

1. Create a RAW socket
2. Bind the socket to network interface
3. Receive packets continuously
4. Read packet header
5. Display source and distination IP addr
6. Display the protocol used
7. Continue until user stops program

INPUT:

network packets received through the network interface

CODE:

```
import socket
s: socket.socket(socket.AF_INET, socket.SOCK_RAW,
                 socket.INPROTO_IP)
s.bind(("0.0.0.0",0))
s.setsockopt(socketIPPROTO_IP, Socket-IP_HDRINCL,1)
print("Packet sniffer started...")
while true:
    packet, addr = s.recvfrom(65565)
    ip-header = packet[0:20]
    iph = struct.unpack("!BBHHHBBH4S4S", ipheader)
.Str-ip = socket.inet-ntoa(iph[8])
dist_ip = socket.inet_ntoa(iph[9])
print("source IP:", src_ip)
print("Destination IP:", dist_ip)
print("Packet Length:", len(packet))
```

OUTPUT:

```
Packet sniffer started
Source IP: 192.168.1.10
Destination IP: 192.168.1.1
Packet Length: 79
```

Result:

Thus, a RAW socket program was successfully implemented for capture and display network packets

Exp - 15
Date - 8/10/24

Analyse various types of Server logs using webalizer tool

Aim:

To analyse different types of server logs using webalizer tool

Algorithm:

1. Install and open webalizer
2. Select the web server log file
3. Configure webalizer to read the log file
4. Run webalizer to process the logs
5. Open generated html reports
6. Analyze visitors, requests, pages, data transfer & HTTP status codes.

Input:

The web server log file such as access log

Output:

Webalizer generates reports showing:

* Number of visitors
* Number of requests
* most visited page
* Data transferred
* Browser and operating systems
* HTTP status codes

Result:

The web logs were successfully analysed using webalizer and the web traffic statistics were generated.